

Домашнее задание: Журналирование и восстановление в PostgreSQL

Цель

На практике изучить работу WAL и восстановление базы данных после случайного удаления таблицы.

Задание

Часть 1: Подготовка (5 минут)

1. Создайте тестовую базу данных и таблицу:

Используйте docker:

```
# Запуск контейнера
docker-compose up -d
```

```
# Подключение к PostgreSQL
docker exec -it wal-learning psql -U postgres
```

```
CREATE DATABASE wal_homework;
\c wal_homework
```

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT NOW()
);
```

```
INSERT INTO users (name, email) VALUES
('Alice', 'alice@example.com'),
('Bob', 'bob@example.com'),
('Charlie', 'charlie@example.com');
```

```
SELECT * FROM users;
\q
```

Ожидаемый результат: Таблица с 3 пользователями создана.

Часть 2: Настройка архивирования WAL (5 минут)

2. Создайте директорию для архива и настройте архивирование:

```

# Зайти в контейнер
docker exec -it wal-learning bash

# Создать директорию для архива
mkdir -p /tmp/wal_archive
chmod 700 /tmp/wal_archive

# Изменить владельца на postgres
chown -R postgres:postgres /tmp/wal_archive

```

3. Настроить параметры архивирования:

```

# Подключиться к БД
psql -U postgres wal_homework

-- Включить архивирование
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command = 'cp %p /tmp/wal_archive/%f';

-- Установить таймаут для быстрого архивирования (для тестирования)
ALTER SYSTEM SET archive_timeout = '30s';

-- Применить настройки
SELECT pg_reload_conf();

\q

```

4. Перезапустить PostgreSQL для применения изменений:

```

# Выйти из контейнера
exit

# Перезапустить контейнер
docker-compose restart

# Зайти обратно в контейнер
docker exec -it wal-learning bash

```

5. Проверить архивирование:

```

# Подключиться к БД
psql -U postgres wal_homework

-- Принудительно переключить WAL для архивирования
SELECT pg_switch_wal();

-- Добавить данные для теста
INSERT INTO users (name, email) VALUES ('David', 'david@example.com');

-- Снова переключить WAL
SELECT pg_switch_wal();

```

\q

Проверить, что WAL файлы появились

```
ls -lh /tmp/wal_archive/
```

Проверить статистику архивирования

```
psql -U postgres wal_homework -c "SELECT * FROM pg_stat_archiver;"
```

Ожидаемый результат: - В /tmp/wal_archive/ должны появиться WAL файлы (по ~16MB каждый) - archived_count в pg_stat_archiver должен быть > 0

Часть 3: Создание базового backup (5 минут)

6. Создайте базовый backup:

Убедитесь, что вы находитесь в контейнере (после шага 5)

Переключиться на пользователя postgres

```
su postgres
```

Создать директорию для backup

```
mkdir -p /tmp/pg_backup
```

Создать базовый backup

```
pg_basebackup -D /tmp/pg_backup -Fp -Xs -P
```

Проверить размер

```
du -sh /tmp/pg_backup/
```

Выйти из пользователя postgres (останетесь в контейнере как root)

```
exit
```

Ожидаемый результат: Backup создан, размер примерно 40-50MB.

Часть 4: Добавление данных и “авария” (5 минут)

7. Запишите текущее время и удалите таблицу:

Подключиться к БД

```
psql -U postgres wal_homework
```

-- Записать текущее время (запомните это время!)

-- Скопируйте вывод в формате: YYYY-MM-DD HH:MM:SS

```
SELECT to_char(now(), 'YYYY-MM-DD HH24:MI:SS');
```

Подождите 5-10 секунд

-- "Авария" - удаление таблицы

```
DROP TABLE users;
```

```
-- Проверить, что таблица удалена
\dt

-- Попытка выбрать данные (должна быть ошибка)
SELECT * FROM users;

\q

# Выйти из контейнера
exit
```

Ожидаемый результат: Таблица users удалена, ошибка “relation does not exist”.

Часть 5: Восстановление через PITR (15 минут)

В Docker контейнере PostgreSQL запускается как PID 1, поэтому используем специальный метод для Docker.

8. Подготовьте данные для восстановления:

```
# Зайти в контейнер
docker exec -it wal-learning bash

# Переключиться на postgres
su postgres

# Скопировать backup в отдельную директорию для восстановления
rm -rf /tmp/pg_backup_for_restore
cp -r /tmp/pg_backup /tmp/pg_backup_for_restore

# Создать файл recovery.signal
touch /tmp/pg_backup_for_restore/recovery.signal
```

9. Настроить параметры восстановления:

Способ 1 (для тех, кто знаком с vi):

```
# Отредактировать postgresql.auto.conf
vi /tmp/pg_backup_for_restore/postgresql.auto.conf
```

В редакторе vi: 1. G (перейти в конец файла) 2. o (создать новую строку) 3. Добавьте эти строки (замените время на ВРЕМЯ_ПЕРЕД_DROP из шага 7):

```
restore_command = 'cp /tmp/wal_archive/%f %p'
recovery_target_time = '2025-12-08 17:04:56'
recovery_target_action = 'promote'
```

4. Нажмите Esc, затем :wq и Enter

Способ 2 (альтернатива через echo):

```
# Добавить параметры восстановления (замените время на ВРЕМЯ_ПЕРЕД_DROP из шага 7)
echo "restore_command = 'cp /tmp/wal_archive/%f %p'" >> /tmp/pg_backup_for_restore/postgresql.conf
echo "recovery_target_time = '2025-12-08 17:04:56'" >> /tmp/pg_backup_for_restore/postgresql.conf
echo "recovery_target_action = 'promote'" >> /tmp/pg_backup_for_restore/postgresql.conf
```

Проверьте результат:

```
cat /tmp/pg_backup_for_restore/postgresql.auto.conf
```

```
# Выйти из пользователя postgres
exit
```

Ожидаемый результат: В конце файла должны быть добавлены 3 строки с параметрами восстановления.

10. Выполнить восстановление:

```
# Выйти из контейнера на хост
exit
```

```
# На хосте: скопировать подготовленные данные наружу
docker cp wal-learning:/tmp/pg_backup_for_restore ./restore_data
```

```
# Остановить контейнер
docker-compose stop
```

```
# Заменить данные через временный контейнер
# Эта команда автоматически определит имя volume
docker run --rm -v $(basename $(pwd))_pgdata:/data -v $(pwd)/restore_data:/restore_data \
  rm -rf /data.bak \
  cp -r /data /data.bak \
  rm -rf /data/* \
  cp -r /restore/* /data/ \
  chown -R 70:70 /data
"
```

```
# Запустить контейнер
docker-compose up -d
```

```
# Подождать 15 секунд для завершения восстановления
sleep 15
```

11. Проверить восстановление:

```
# Проверить данные
docker exec -it wal-learning psql -U postgres wal_homework -c "SELECT * FROM users"
```

Ожидаемый результат: Все пользователи (Alice, Bob, Charlie, David) восстановлены, DROP TABLE не применился!

Cleanup (при необходимости)

Остановить и удалить контейнер

```
docker stop wal-learning
```

```
docker rm wal-learning
```

Полная очистка (удалит все данные)

```
docker-compose down -v
```

```
rm -rf restore_data
```

Очистка системы Docker (опционально)

```
# docker system prune -a -f
```

Вопросы для ответа

1. Сколько WAL файлов было создано в /tmp/wal_archive/?
 2. Какой размер базового backup в /tmp/pg_backup/?
 3. Как PostgreSQL определил, до какого момента восстанавливать данные?
 4. Что произойдет, если указать recovery_target_time после DROP TABLE?
-

Замечания

1. **Права доступа:** Директория /tmp/wal_archive/ должна принадлежать пользователю postgres, иначе архивирование не будет работать.
2. **archive_timeout:** Для продакшена не нужно устанавливать маленький таймаут. Мы установили 30 секунд только для быстрого тестирования.
3. **Docker специфика:** В Docker контейнере PostgreSQL запускается как PID 1, поэтому стандартные методы остановки/запуска через pg_ctl не работают.
4. **recovery_target_time:** Указывайте время ПЕРЕД нежелательной операцией (DROP TABLE), а не после. Точность восстановления зависит от архивирования WAL.
5. **Backup стратегия:** В продакшене используйте специализированные инструменты типа pgBackRest, Barman или pg_probackup для более надежного бэкапа и восстановления.
6. **Команда su:** Если команда su postgres не работает, попробуйте su - postgres. В разных образах Alpine поведение может отличаться.